

DSCC : Dynamic synergistic congestion control for lossless RDMA datacenter networks

Jianxing Zhuge , Zeming Gao , Ye Tian *, Jun Wang , Shaoxuan Yun ,
Xiangyang Gong *

State Key Laboratory of Networking and Switching Technology, Beijing University of Posts and Telecommunications, Beijing, China

ARTICLE INFO

Keywords:

RDMA
PFC
ECN
Datacenter networks

ABSTRACT

Large-scale incast traffic generated by AI training tasks presents significant challenges to RDMA networks. Existing congestion control mechanisms, such as Artificial Intelligence ECN (AI ECN) and fixed threshold priority flow control (PFC), are often ineffective in mitigating frequent PFC pauses caused by ECN's delayed feedback. This issue is further aggravated by the static PFC triggering mechanism in commercial switches, where pause frames are triggered once the queue length reaches a fixed proportion (typically 11.11%) of the remaining buffer. This leads to premature transmission stops and inefficient buffer utilization. To address these limitations, this paper proposes DSCC, a dynamic and synergistic congestion control algorithm that combines PFC and ECN. Building upon the AI ECN framework, DSCC adjusts the PFC threshold according to the degree of incast while adhering to the PFC and ECN threshold constraints. It dynamically tunes both the growth rate and triggering threshold of PFC in response to the current congestion level at the switch. By adaptively increasing the PFC threshold, DSCC reduces the frequency of PFC-induced pauses and improves buffer and bandwidth utilization. Simulation results in ns-3 demonstrate that under incast traffic scenarios, DSCC achieves up to 19.2% reduction in flow completion time (FCT) compared to AI ECN.

1. Introduction

In recent years, the proliferation of intelligent computing workloads, such as large-scale machine learning, data analytics, and real-time inference, has given rise to increasingly frequent and massive incast events within data center networks (DCNs). In particular, during distributed training of deep neural networks or other large models, the worker nodes must periodically exchange gradient updates and snapshots of parameters [1]. This process concentrates traffic toward a small set of parameter servers or aggregation points, producing a deluge of synchronized N-to-1 burst flows. Such concentrated traffic patterns place enormous stress on RoCEv2-based RDMA infrastructures, where congestion and packet drops can quickly escalate. Because RoCEv2 operates without per-flow connections at the network layer, any packet loss immediately triggers costly end-to-end retransmissions, sharply reducing effective throughput and increasing latency [2].

To achieve truly lossless transport under these adverse conditions, RoCEv2 typically relies on two mechanisms: Explicit Congestion Notification (ECN), which provides end-to-end congestion signaling, and Priority Flow Control (PFC), which prevents packet loss by pausing specific traffic classes at the link layer. When carefully calibrated to work

in tandem, ECN and PFC can mitigate both transient and persistent congestion, ensuring high-performance lossless communication even in the face of sudden large-scale incast bursts [3].

However, traditional ECN mechanisms with static thresholds cannot simultaneously achieve low latency and high throughput in modern, highly dynamic data center environments [4]. To address this limitation, researchers have proposed adaptive approaches, including Artificial Intelligence ECN (AI ECN) algorithms such as ACC [5] and QueuePilot [6], which dynamically adjust thresholds to traffic patterns. Nonetheless, these solutions often overlook the critical interaction between ECN and PFC, which remains a key challenge in preventing excessive pause events and ensuring stable performance. In particular, ECN often reacts too slowly to sudden incast bursts, failing to alleviate congestion in time [7]. This leads to frequent activation of PFC, excessive link pauses, and even deadlocks.

As shown in Fig. 1, in an N-to-1 scenario, if the ECN threshold at the egress port is higher than the sum of all upstream PFC thresholds, upstream queues will trigger PFC pause frames before ECN marking occurs. To ensure proper control behavior, the total PFC pause threshold at ingress must exceed the ECN threshold at egress, so that flows are throttled before being paused. However, commercial switches often trig-

* Corresponding authors.

E-mail addresses: yetian@bupt.edu.cn (Y. Tian), xygong@bupt.edu.cn (X. Gong).

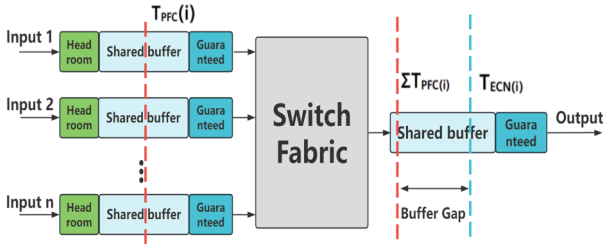


Fig. 1. Lossless queue buffer in N-to-1 incast scenarios.

ger PFC when queue occupancy exceeds a fixed 11.11% of the remaining buffer [8,9]. This static configuration fails to consider actual traffic intensity or adapt to changing ECN thresholds, making it inadequate for handling large-scale burst traffic. On the one hand, an overly conservative threshold may trigger premature PFC pause frames-causing underutilized buffers, wasted link capacity, and unnecessary flow stalling; on the other hand, aggressive or excessive PFC activation can induce head-of-line blocking, propagate congestion to upstream nodes, and in extreme cases lead to cascading network deadlocks [10].

These limitations highlight the need for a unified and adaptive congestion control framework that can dynamically coordinate ECN and PFC behavior in real time. To address this, we propose DSCC, a dynamic and synergistic congestion control algorithm that extends AI ECN with PFC-ECN constraint awareness. DSCC maintains the necessary relationship between the two thresholds under joint adjustment to ensure effective flow control.

Designing such an algorithm involves several key challenges:

(1) **Pause tuning:** How to accurately sense and adapt to the real-time growth rate of PFC pause frames under varying incast degrees, enabling controlled tolerance of more extreme bursty traffic during prolonged high-intensity events and ensuring ultra-fast PFC triggering during brief, high-frequency surges. (2) **Buffer balancing:** How to maintain an appropriate buffer gap during dual ECN and PFC threshold adjustments while avoiding packet loss at the egress queue. (3) **Adjustment stability:** How to fine-tune the frequency of PFC threshold changes to prevent link turbulence due to overreaction, without being too slow to track network dynamics.

To address these challenges, we propose DSCC, a dynamic coordination algorithm based on deep reinforcement learning. DSCC adopts a deep Q-learning model to jointly optimize PFC and ECN thresholds. By continuously monitoring traffic characteristics, DSCC applies high ECN thresholds for long-lived, high-throughput flows, and low ECN thresholds for short, latency-sensitive flows, achieving a balance between latency and bandwidth utilization. In parallel, DSCC adjusts the PFC threshold by observing ECN trigger patterns and estimating the incast degree via the number of concurrent senders and receivers at each switch port. The PFC growth rate is then adaptively tuned, and threshold update frequency is regulated based on queue trends, preventing link instability. Moreover, DSCC enforces PFC-ECN coordination constraints to maintain a safe buffer gap and prevent packet loss at egress ports.

The key contributions of this paper are as follows:

- We analyze the limitations of existing PFC adjustment strategies, such as static thresholding, coarse-grained tuning, slow convergence, and introduce a novel, AI-assisted dynamic PFC adjustment mechanism that adapts thresholds at sub-RTT timescales and on a per-flow basis.
- We design and implement DSCC, a coordinated congestion control framework that jointly adjusts PFC pause thresholds and ECN-marking parameters in real time, leveraging machine-learned models to predict imminent congestion and optimize buffer utilization in modern data center topologies.

- We evaluate DSCC through ns-3 simulations. Results show that under incast-dominated traffic patterns, DSCC reduces flow completion time (FCT) by up to 19.2% compared to existing AI ECN schemes.

2. Related work

Congestion control remains a core challenge in computer networks and distributed systems. The demand for ultra-low latency and high throughput, particularly in data center networks (DCNs), has driven extensive research into efficient, adaptive control mechanisms.

DCQCN: Classic ECN-Driven Control. DCQCN [6] represents a foundational approach to congestion control in RDMA networks, built upon ECN for feedback-driven sender rate modulation. It categorizes network conditions into three congestion states and adapts the transmission rate accordingly to maintain stable performance. A key feature of DCQCN is its coordination strategy, where PFC is only triggered after ECN marks are observed, thus reducing the risk of premature link pauses and head-of-line blocking. Despite its effectiveness under moderate loads, DCQCN faces notable limitations. Its convergence process is relatively slow, especially during sudden congestion shifts, and it depends heavily on manually tuned parameters, which may not generalize well across varying workloads. Moreover, in high-intensity incast scenarios, if the PFC threshold is set too high, the delay in triggering PFC can result in buffer overflow and packet loss, undermining RDMA's lossless transport guarantees. Crucially, DCQCN employs static configuration for both PFC and ECN thresholds, lacking a dynamic mechanism to adjust the buffer gap between them. This rigid coupling often leads to a mismatch where ECN signals are generated too late to effectively throttle sources before the static PFC limit is breached, failing to achieve the synergistic control required for lossless transmission under bursty traffic.

Telemetry-Assisted and Proactive Designs. To overcome the latency and responsiveness limitations of ECN-based schemes, telemetry-driven solutions have emerged. HPCC [2] introduces in-band network telemetry (INT), embedding per-packet feedback such as queue lengths and round-trip time (RTT) directly into the data plane. This enables ultra-fast, sub-RTT congestion response without the need for iterative rate adjustments. Building upon this idea, PACC [7] enhances proactive congestion handling by predicting congestion before it fully manifests. It leverages downstream telemetry data to pre-generate and send Congestion Notification Packets (CNPs), thereby achieving earlier and more aggressive sender back-off. FCC [8] further pushes this direction by using ultra-lightweight switch-side counters to detect microbursts and enforce source throttling within microseconds. While these approaches offer exceptional precision and responsiveness, their practical deployment is constrained by hardware dependencies, particularly the need for programmable switches with telemetry and counter support, which are not yet ubiquitous across production-scale datacenter networks. Furthermore, these telemetry-based solutions primarily focus on optimizing source-side rate control while treating switch-side buffer management (PFC) as a static fail-safe. They do not actively adjust switch PFC thresholds in coordination with the telemetry feedback, leaving the network vulnerable to uncoordinated pausing when source throttling cannot react instantaneously to extreme microbursts.

Learning-Based and AQM-Driven Methods. Recent trends have also explored the use of intelligent queue management and machine learning to drive congestion decisions. ACC [9] and BBQ [10] are notable examples that adopt closed-loop controllers to dynamically adjust ECN thresholds based on queue occupancy or predictive buffer usage patterns. These methods remove the need for manual threshold tuning and improve system stability under relatively stable workloads. However, they typically operate independently of PFC and are less effective in mitigating congestion during burst-heavy or incast-intensive traffic. QueuePilot [11] introduces reinforcement learning to fine-tune ECN behavior at short time intervals, reducing tail latencies and improving flow completion time (FCT), particularly in environments with constrained

buffer sizes. R-PFC [12] applies a decision-tree classifier to make fine-grained PFC triggering decisions, balancing between responsiveness and control overhead. Meanwhile, LSCC [13] introduces hierarchical segmentation of network paths, allowing local learning agents to optimize congestion decisions more efficiently. Despite their intelligent adaptability, these AQM-based methods often address either ECN or PFC individually, lacking a cohesive framework to unify both mechanisms for holistic congestion control. Specifically, they fail to dynamically regulate the relative offset (safety margin) between PFC and ECN thresholds. Without joint optimization, the system cannot guarantee that congestion notification optimization precedes link pausing, leading to either resource underutilization or frequent, disruptive PFC storms.

Cross-Datacenter and Hierarchical Control. To scale congestion control beyond the confines of a single data center, hierarchical control architectures such as BCC [14] have been proposed. These schemes divide responsibility between localized (rack-level or pod-level) controllers and global coordinators, enabling efficient handling of wide-area congestion and inter-datacenter RDMA traffic. By incorporating feedback loops at multiple levels, they achieve scalable and coordinated control across heterogeneous network segments. However, the multi-layered design introduces additional control latency and complexity, which may be less suitable for fine-grained, low-latency congestion response within a single data center. Additionally, their hierarchical nature makes them more suitable for long-flow or bulk data transfers, rather than the short, bursty incast flows common in modern AI and storage workloads. Moreover, the coarse-grained, high-latency control loops inherent in these hierarchical architectures make them ill-suited for the microsecond-level synchronization required between PFC and ECN. They cannot react quickly enough to transient switch buffer dynamics to prevent the uncoordinated activation of pause frames common in intra-datacenter AI incast scenarios.

In summary, while existing approaches to congestion control offer valuable capabilities, they each focus on isolated aspects of the problem. These methods typically either emphasize ECN signaling without considering the dynamics of PFC, or optimize control policies under assumptions of stable buffer availability and moderate traffic patterns. However, none of them fully address the need for tightly coupled, dynamic coordination between ECN marking and PFC pause mechanisms, especially in the face of increasingly bursty incast traffic and shrinking switch buffer sizes common in modern data centers. However, effective coordination between ECN and PFC, especially under constrained buffer settings and N-to-1 traffic burst, remains an open problem.

3. Background and motivation

3.1. Background

Remote Directly Memory Access (RDMA). Traditional data centers typically rely on the TCP/IP protocol stack for data transmission. However, TCP/IP introduces high latency and significant CPU overhead, rendering it unsuitable for high-performance computing applications. RDMA addresses these limitations by enabling direct memory access between endpoints without kernel involvement, thereby ensuring high throughput, minimal latency, and efficient CPU utilization [15]. Among RDMA implementations, RoCE (RDMA over Converged Ethernet) encapsulates RDMA packets in UDP, but packet loss necessitates end-to-end retransmissions at the application layer, degrading performance. Consequently, lossless operation in RDMA networks is achieved by integrating link-layer flow control, such as Priority Flow Control (PFC), with network-layer congestion control, such as Explicit Congestion Notification (ECN).

Explicit Congestion Notification (ECN). ECN provides end-to-end congestion signaling by marking packets before buffers overflow. When the egress queue length surpasses the minimum threshold, switches begin probabilistic marking using the parameter p . If queuing persists and the length exceeds $\frac{1}{p}$, marking occurs with maximum probability.

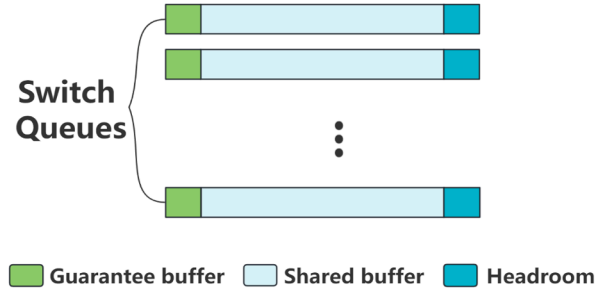


Fig. 2. Buffer allocation mode of shared buffer switch.

Receivers that obtain ECN-marked packets issue Congestion Notification Packets (CNPs) back to senders, prompting immediate rate adjustment and avoiding packet loss and retransmission overhead. By preserving packet delivery while signaling impending congestion, ECN improves overall network utilization and latency.

Priority Flow Control (PFC). PFC extends the standard Ethernet pause mechanism by supporting up to eight priority-based virtual channels per link [16]. Rather than halting all traffic on a congested port, PFC can selectively pause individual traffic classes, enabling zero-loss service for latency-sensitive flows while allowing other traffic to continue. In practice, RoCE deployments often configure only two priority classes. Despite its efficacy, excessive PFC activation can introduce head-of-line blocking, congestion propagation, and deadlock risks, so PFC should be triggered sparingly and in coordination with ECN.

Shared Buffer Switch Architecture. As shown in Fig. 2, modern Ethernet switches employ shared buffer pools to maximize link utilization. The total buffer space is partitioned into three regions: guaranteed buffer (per-queue reservation, typically minimal), shared buffer (dynamically allocated among active queues), and headroom (buffer reserved for in-flight packets during pause periods). Headroom is usually dimensioned as the product of link bandwidth and round-trip time (RTT). Since the guaranteed buffer remains largely unused, the effective PFC threshold is set at the boundary between shared buffer and headroom. Fig. 3 Under the default NIC vendor configuration, the PFC pause-off threshold is calculated as: $Xoff = \alpha(1 + \alpha) \times Bufferfree$, where α is a constant (typically 1/8) and $Bufferfree$ is the remaining buffer size [2,3]. The relationship indicates that the PFC threshold is inversely related to the amount of shared buffer in use; as the queue length increases, the available buffer decreases, causing the PFC threshold to drop. Calculations show that the PFC will be triggered when the ingress queue exceeds more than 11.11% of the available buffer.

Deep Reinforcement Learning. The dynamic coordination of PFC and ECN can be characterized as a deep reinforcement learning (DRL) problem [17]. DRL leverages deep neural networks to handle high-dimensional state and action spaces and employs reinforcement learning to learn policies that maximize long-term rewards. Compared to traditional model-based methods, DRL is better suited to the dynamic, non-linear relationships inherent in network congestion control [18], such as interactions among queue length, traffic rates, and ECN/PFC trigger frequencies, without requiring oversimplified assumptions or complex mathematical modeling [19].

3.2. Motivation

PFC and ECN complement each other in the congestion control algorithm and work together. In the switch, the ECN mark is often first triggered to slow down the source end, and then the PFC is triggered to suspend the transmission of traffic hop by hop. Therefore, the PFC threshold is often set to be triggered after ECN [20].

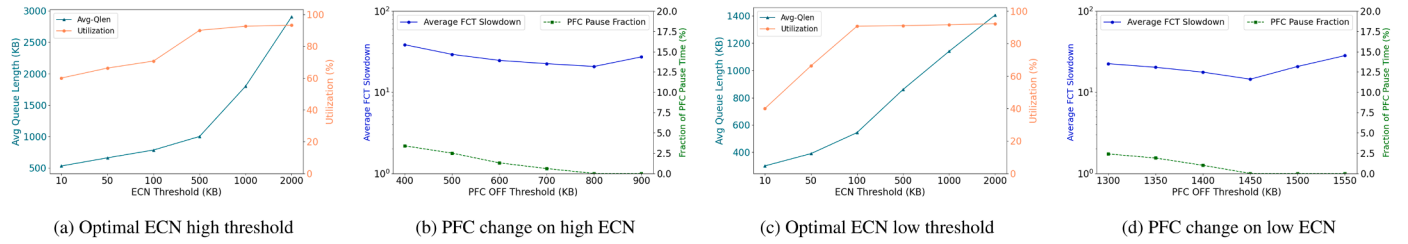


Fig. 3. The optimal ECN and PFC thresholds for different scenarios.

3.2.1. Necessity of coordinated ECN-PFC threshold adjustment

In the N-to-1 traffic scenario, we firmly believe that the following critical constraints need to be precisely met between the PFC and ECN thresholds [21], Specifically:

ECN-PFC Buffer Gap. The buffer interval between ECN marking and PFC activation must be sufficient to allow ECN's feedback to throttle sources before PFC pauses occur. A proper gap reduces unnecessary PFC events [22].

Threshold Sum Condition. The aggregate PFC thresholds of upstream queues must exceed the downstream ECN threshold, ensuring that throttling (via ECN) precedes any PFC-induced pausing. This prevents premature PFC and excessive CNP generation under high-volume bursts.

Downstream Buffer Guarantee. The downstream queue's dynamic buffer threshold must exceed the sum of all upstream PFC thresholds plus headroom, preventing packet loss before upstream PFC flow control engages [23].

3.2.2. Inefficiency of static PFC thresholds

These interdependencies underscore the need for coordinated, dynamic adjustment of ECN and PFC thresholds rather than static configurations. However, realizing such coordination is non-trivial, especially because existing PFC thresholds are often configured statically and inefficiently, without regard to traffic dynamics or downstream buffer constraints [24].

The existing algorithms all use PFC pauses triggered at fixed positions. Taking the existing Ethernet switch as an example, triggering PFC pauses when more than 11.11% of the available buffer will waste a lot of buffer resources. Therefore, the PFC threshold can be dynamically raised from its existing setting. For different degrees of many-to-one scenarios, when the incast degree is small, we make the PFC adjustment range smaller to save resources and costs. When the incast degree is large, we make the PFC adjustment range larger. It can tolerate more burst flows and avoid the triggering of PFC pauses faster due to short-term traffic surges. However, the PFC threshold should not be set too high to ensure that the buffer space of the downstream queue is sufficient before all upstream PFC flow control is triggered, so that no packet loss occurs. Therefore, we should dynamically adjust the proportion of the buffer occupied by the PFC-triggered pause frame to adapt to the complex and changing data center network environment [25].

Adjustment Space: In N-to-1 incast scenarios, static thresholds often trigger PFC prematurely, leading to underutilized buffers. DSCC expands this space by adaptively raising the PFC threshold when it senses a high incast degree, allowing the network to absorb more burst traffic without immediate pauses.

Opportunities for DSCC: DSCC is most effective during sudden, high-intensity incast bursts where ECN feedback is too slow to throttle sources. By sensing the queue growth rate and incast degree, DSCC identifies opportunities to delay PFC activation just enough for ECN markers to take effect, thereby reducing unnecessary link pauses while maintaining losslessness.

3.3. Experimental analysis

To validate our assumptions regarding threshold coordination, we selected two representative incast scenarios with varied burst patterns. We built a small spine-leaf topology using ns-3. 60 server nodes are connected by a switch. The link bandwidth is 100Gbps and the latency is 1us. We change the size of ECN, then select the best ECN threshold point, record and increase the corresponding PFC pause frame threshold at this time, and record the corresponding FCT and PFC trigger time ratio. In this experiment, we selected several discrete threshold points of ECN and PFC at equal intervals by manual sampling. The best threshold is selected by comparing the corresponding indicators between a limited number of discrete points. Key performance metrics such as average queue length, bandwidth utilization, average flow completion time and PFC trigger time ratio were used to assess the effectiveness of ECN-PFC coordination. It should be noted that the PFC threshold in this experiment is the sum of the PFC thresholds of all upstream queues [26]. We simulated two scenarios:

- **Scenario 1(5-1 incast, 3MB bursts):** 5 senders transmit large flows to one receiver. Optimizing for throughput requires tolerating moderate queue to maximize bandwidth.
- **Scenario 2(50-1 incast, 500KB bursts):** 50 senders generate microbursts to one receiver. Rapid congestion feedback is critical to prevent buffer overflow.

Observation #1: The results show that under different many-to-one and burst flow sizes, by changing the ECN threshold, the queue length and bandwidth utilization can be balanced, ensuring high bandwidth utilization under the condition of low queue length. For example, in Fig. 3a, in the high-throughput long-flow scenario, the optimal point of ECN is the high threshold of 500KB, and in Fig. 3c, in the high concurrency and short-flow scenario, the optimal point of ECN is the low threshold of 100KB.

Observation #2: At the optimal ECN threshold, increasing the PFC threshold based on the existing PFC threshold can reduce FCT and reduce the frequency of PFC activation, as shown in Fig. 3b and d. At the same time, by comparing Fig. 3b and d, it can be concluded that when the many-to-one level is more serious, the increase in PFC should be greater. As shown in Fig. 3b, the existing PFC threshold is the optimal PFC threshold shown in this experiment, 1200KB, which is 475KB higher than the default threshold. In Fig. 3d, the optimal PFC threshold shown in this experiment is 1450KB, which is only 187KB higher than the default threshold. When the threshold increases, although the PFC trigger frequency decreases, the FCT increases, which means that the PFC threshold amplitude should not be adjusted too large, and the restriction relationship with ECN needs to be considered, otherwise it will be detrimental to the performance of the network.

4. Design overview

In modern data center networks, traffic patterns often exhibit varied inflow-outflow ratios, especially in many-to-one (N:1) scenarios. Traditional congestion control methods, with ECN thresholds tuned via single-objective learning and PFC triggered at fixed buffer occupancy

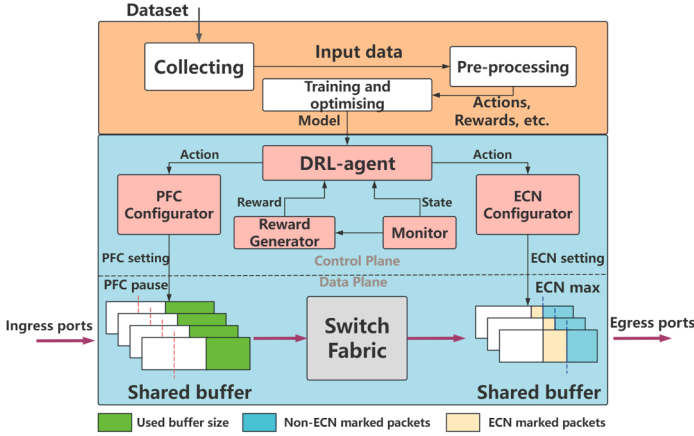


Fig. 4. Overview design of DSCC.

ratios, fail to capture the coupled dynamics of ECN and PFC, leading to suboptimal buffer utilization and resource waste [27]. To address these limitations and to ensure agile, coordinated control, we propose DSCC: a DRL based Synchronized Congestion Control algorithm. DSCC retains the adaptive marking strategy of AI ECN and extends it with an AI-driven PFC policy, enabling precise flow-control matching across diverse traffic modes and significantly improving network efficiency and buffer utilization [28].

We model the environment as a Markov Decision Process (MDP), which is a mathematical framework that represents how an intelligent agent interacts with a dynamic environment, influencing its state, and earning rewards by selecting various actions. By evaluating possible actions and subsequent rewards in each state, MDP aims to find an optimal strategy that allows the agent to obtain the maximum cumulative reward in the long run.

In real-world deployment, DSCC is designed to operate in the Control Plane with a coarse-grained time slot (e.g., control epochs of 10ms or greater), rather than a per-packet adjustment. Modern switching ASICs (such as Broadcom Trident/Tomahawk series or Ruijie switches mentioned in the paper) support “Watermark” features or dynamic buffer profiles that can be updated via SDK APIs. DSCC calculates the optimal threshold and updates these profiles only when significant traffic pattern shifts are detected (filtered by a hysteresis mechanism to prevent oscillation), thereby balancing responsiveness with hardware configuration costs.

4.1. Framework

Fig. 4 describes a congestion control framework based on DRL that optimizes the performance of data center networks by intelligently adjusting PFC (priority flow control) and ECN (explicit congestion notification) policies. The overall architecture is divided into two levels: the data plane and the control plane.

In the data plane of a modern leaf-spine switch, all packets arriving at various ingress ports are placed into a common pool of shared buffer memory. Under typical conditions, buffer occupancy remains low and packets experience minimal queuing delay. However, when multiple high-speed flows converge, such as during incast events, shared buffers can fill rapidly, leading to substantial queue build-up and elevated latency. When congestion occurs, the PFC mechanism can trigger a flow control pause (PFC pause), while the ECN mechanism can mark traffic exceeding the threshold (ECN max).

In the control plane, the DRL agent makes decisions based on the network status collected from the monitor, such as buffer occupancy, throughput, and the number of ECN marked packets. The DRL agent sends adjustment policies to the PFC and ECN configurator to optimize the PFC and ECN settings, thereby dynamically controlling the conges-

Table 1
Status statistics.

ECN _c	The current ECN settings (KB)
qlen _c	The current queue length (KB)
qlen	The average queue length (KB)
N _s	The average ingress rate (KB)
N _r	The average egress rate (KB)
ECN_Ratio	The ratio of packets marked with ECN
PFC _c	The current PFC settings (KB)
q(t)	The current queue growth rate

tion feedback mechanism. At the same time, the reward generator calculates the reward value and is used to optimize the DRL agent policy model to achieve better network performance. The entire system continuously improves decision-making results through training and optimization, increases datacenter network performance, and reduces congestion losses.

5. Algorithm design

In this section, we present the overall design of DSCC, focusing on its core DRL-based decision process. We first provide an overview of the algorithmic workflow, including how the agent observes network states, selects actions, and receives rewards. We then formally describe each component in detail, covering the state space, action space, reward design, and constraint penalties.

5.1. State

In reinforcement learning, the state s represents the observation of the environment, all or part of the information presented by the environment at a certain moment, and is an important basis for the agent to judge the current state of the environment. The state can be a vector that contains various observation data of the environment. In this article, the state is a comprehensive description of the current state of the network operation. It consists of a set of key indicators which can be obtained by monitoring existing switches, as shown in Table 1. In this article, we use the history state of the time window of length $k = 3$ as input, which satisfies the Markov property.

5.2. Action

An action a is a decision made by the agent according to the policy $\pi(a|s)$. In DSCC, the action mainly manifests itself as the adjustment of PFC and ECN parameters. At each decision epoch, the DSCC agent selects an action a_t to adjust the thresholds for ECN and PFC. The action is defined as

$$a_t = (a_{ECN}, a_{PFC}, \delta_t), \quad (1)$$

where:

- $a_{ECN} \in \{\text{Increase, Decrease, NoChange}\}$ is the adjustment of the ECN threshold.
- $a_{PFC} \in \{\text{Increase, Decrease, NoChange}\}$ is the adjustment of the PFC threshold.
- δ_t is a modulation factor applied to the adjustment of the PFC.

The PFC threshold is adjusted only if the ECN trigger frequency exceeds a predefined threshold f_{ECN}^{th} . Mathematically, the PFC threshold adjustment $\Delta T_{PFC}(t)$ is defined as

$$\Delta T_{PFC}(t) = \begin{cases} \delta_t, & \text{if } f_{ECN}(t) > f_{ECN}^{th}, \\ 0, & \text{otherwise,} \end{cases} \quad (2)$$

or equivalently,

$$\Delta T_{PFC}(t) = \delta_t \cdot \mathbb{I}\{f_{ECN}(t) > f_{ECN}^{th}\}, \quad (3)$$

where $\mathbb{I}\{\cdot\}$ is the indicator function.

The modulation factor δ_t for the adjustment of the PFC threshold is determined by the degree of arrival $I(t)$ and the variation rate in the queue $\Delta q(t)$. This can be expressed as

$$I(t) = \frac{N_s}{N_r} \quad (4)$$

$$\delta_t = k_M I(t) \cdot (1 + k_{\Delta q} \Delta q(t)), \quad (5)$$

where k_M and $k_{\Delta q}$ are scaling constants. In this formulation,

- $k_M I(t)$ represents the growth amplitude, ensuring that a higher incast degree $I(t)$ results in a greater adjustment.
- $1 + k_{\Delta q} \Delta q(t)$ represents the adjustment frequency factor, so that a higher queue variation rate $\Delta q(t)$ leads to more frequent adjustments.

Thus, the updated PFC threshold is given by

$$T_{PFC}(t+1) = T_{PFC}(t) + \Delta T_{PFC}(t). \quad (6)$$

5.3. Reward

The reward r is the feedback provided by the environment, which is used to evaluate the pros and cons of the current action. The reward value is usually related to the target task and is used to adjust the strategy later so that the agent can maximize the cumulative reward. In this article, the reward function is designed to guide the DSCC algorithm in achieving the following long-term goals by adjusting the ECN and PFC thresholds.

- Encourage the system to transmit data efficiently and make full use of available bandwidth. By dynamically adjusting ECN and PFC thresholds in response to real-time traffic patterns, the mechanism minimizes idle link periods and smooths throughput across competing flows.
- Reduce queue backlog: Avoid high latency and packet loss caused by long queues. Maintaining a calibrated buffer headroom through coordinated ECN marking and PFC pauses ensures that queues never grow excessively deep during incast bursts.
- Reduce the frequency of congestion control triggering: Reduce the frequent triggering of PFC and mitigate link instability caused by excessive control. By introducing adaptive hold-off intervals and trend-aware filtering, the system prevents oscillatory behavior while still reacting promptly to genuine congestion events.

At each time step t , the immediate reward R_t is defined as

$$R_t = 0.5U_{BW}(t) - 0.2q_{len}(t) - 0.3N_{PFC}(t) \quad (7)$$

where

- $U_{BW}(t)$ is the normalized utilization of the bandwidth at time t .
- $q_{len}(t)$ is the current queue length.
- $N_{PFC}(t)$ is the number of PFC triggers within a time interval.

This function can be seen as a trade-off. The goal is to maximize R_t by increasing $U_{BW}(t)$ while minimizing the penalties incurred by long queues and excessive PFC/ECN events.

Furthermore, the long-term objective is to maximize the expected discounted cumulative reward

$$J = \mathbb{E} \left[\sum_{t=0}^{\infty} \gamma_d^t R_t \right], \quad (8)$$

where $\gamma_d \in (0, 1)$ is the discount factor.

6. Implementation

In this section, we present the complete implementation of DSCC, which is realized through a tightly integrated co-simulation environment combining the ns-3 network simulator [29] and a Python-based Deep Q-Learning (DQL) agent via the ns3-ai framework [30]. This hybrid simulation approach allows for real-time data exchange between the network simulation and the learning model, enabling the agent to observe network states and make adaptive decisions during runtime. We begin by describing the overall system architecture, including the interaction between ns-3 and the DQL agent, the flow of control signals and feedback, and the roles of each module in the learning loop. Following that, we delve into the design of the deep neural network, outlining the structure of the input state space, the action space for threshold adjustments, and the reward function used to guide learning. We also elaborate on the training procedure, including experience replay, exploration strategies, and convergence criteria. Finally, we explain how we generate a diverse set of network scenarios, including varying incast degrees, traffic patterns, and buffer configurations. This diversity ensures that the learned policy generalizes well to real-world environments and remains robust across different congestion levels and workload intensities.

Furthermore, we outline the training procedure, where the agent executes a learning episode consisting of multiple simulation steps, gathers experience tuples, and periodically updates its policy to improve decision-making. We explain how convergence is monitored and how the learned Q-network is deployed for online inference in ns-3 [31]. Finally, we discuss how we constructed diverse and randomized network scenarios-including variations in topology, link bandwidth, delay, packet loss, and buffer configurations-to improve DSCC's generalization ability and robustness across different traffic conditions [32].

6.1. Integration with ns-3 simulation environment

We facilitate real-time interaction between the DSCC learning agent and the simulated network environment using the ns3-ai framework. This framework bridges the C++-based ns-3 simulator and the Python-based DRL agent. The training loop operates in a synchronized manner between ns-3 and Python. In each simulation step, the ns-3 environment collects key runtime network statistics, including queue occupancy, buffer utilization, ECN marking ratios, PFC pause frame counts, and packet drop statistics. These data are serialized and passed to the Python agent via ns3-ai's communication interface as the current state s_t . The Python-based agent, as shown in Fig. 5, implemented using Deep Q-Learning (DQL), selects an action a_t from a predefined discrete action space. The action is transmitted back to ns-3 and immediately applied to the corresponding network elements.

After applying the action, the simulation continues for 10 ms, after which a new state s_{t+1} and a scalar reward r_t are computed. The reward is based on performance objectives such as minimizing flow completion time (FCT), reducing packet loss, and avoiding excessive PFC triggering. This transition tuple (s_t, a_t, r_t, s_{t+1}) is stored in a replay buffer for training the Q-network.

This tightly coupled co-simulation loop, powered by ns3-ai, allows the DSCC model to be trained efficiently and interactively within a high-fidelity network simulation environment.

6.2. Overhead and scalability

The overhead and Scalability is minimized by design.

Monitoring: DSCC utilizes standard switch counters (e.g., queue length, ECN marking ratio, ingress/egress rates) that are already available in modern merchant silicon.

Computational Complexity: The DRL agent in DSCC is intentionally designed to be lightweight in order to minimize computational overhead and facilitate deployment in production-grade switches. Specifically, the policy network adopts a shallow Multi-Layer Perceptron (MLP)

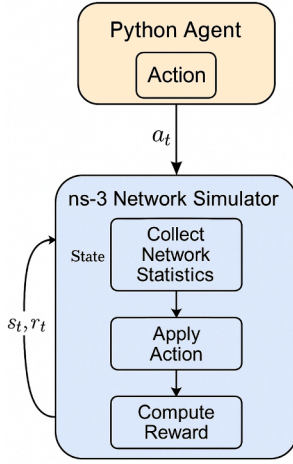


Fig. 5. Training process.

architecture with only three fully connected layers and a limited number of neurons per layer. This compact design ensures that the forward inference involves only a small number of matrix-vector multiplications. As a result, the inference latency is on the order of microseconds, which is several orders of magnitude smaller than the control epoch (10-50 ms) at which congestion control decisions are updated. Consequently, the computational overhead introduced by DSCC is negligible relative to both packet forwarding and queue management operations, and it does not interfere with the timing requirements of high-speed RDMA traffic or switch control planes.

Scalability: DSCC is inherently scalable due to its decentralized deployment architecture. Each switch-particularly Top-of-Rack (ToR) switches-runs an independent DRL agent that makes decisions solely based on local observations, such as instantaneous queue length and incoming traffic rate. This localized decision-making paradigm avoids any form of global state aggregation or cross-switch coordination, ensuring that both the state and action spaces remain constant regardless of the overall network size. As a result, the computational and memory overhead of DSCC does not grow with the number of switches or flows in the data center, preventing the curse of dimensionality commonly encountered in centralized learning-based control schemes. Furthermore, the training process is conducted offline or in a shadow-learning mode, where the learned policy is evaluated without directly affecting active forwarding behavior. This design choice isolates the learning overhead from the data plane, enabling safe deployment and incremental rollout in large-scale data center networks without disrupting ongoing traffic.

6.3. Deep Q-learning process

Neural network design: We use a feedforward neural network (DQN) with two hidden layers of 128 and 64 neurons, respectively, and ReLU activations. The input is the state vector, and the output is a Q-value for each action.

Training Hyperparameters: The model was trained with the following hyperparameters: a learning rate of 1×10^{-4} , a discount factor (γ) of 0.99, and a replay buffer size of 100,000 to ensure stable experience replay. Training was performed using mini-batches of 64 samples, while the target network was updated every 1,000 steps to maintain learning stability. The agent was trained for a total of 10,000 episodes to ensure sufficient convergence.

Training process: During training, the agent interacts with ns-3 under randomly generated network scenarios, including varying topologies, link delays, bandwidth, and packet loss rates. An experience replay buffer stores transitions (s_t, a_t, r_t, s_{t+1}) , and minibatches are sampled to update the network using the Bellman equation. We use the ϵ -greedy strategy for exploration, with ϵ annealed from 1.0 to 0.05 over time.

The target network is updated every 1000 steps to stabilize training. The detailed training process is as follows.

Specific Interaction: DSCC explicitly enforces the precedence of ECN over PFC by introducing a mandatory Buffer Gap constraint between the ECN marking threshold and the PFC pause threshold. This constraint guarantees a non-negligible buffering interval during which congestion can be signaled to end hosts through ECN before any link-layer backpressure is triggered. To operationalize this principle, DSCC embeds the Buffer Gap into the Deep Reinforcement Learning (DRL) reward function: any action that reduces the distance between the ECN threshold and the aggregate PFC threshold beyond a predefined safety margin is penalized. As a result, the learning agent is guided to maintain a stable ordering between congestion signals, preventing ECN from being shadowed or rendered ineffective by premature PFC activation. This design ensures that transport-layer congestion control mechanisms retain sufficient reaction time, thereby mitigating adverse side effects such as head-of-line blocking and congestion propagation caused by excessive PFC pauses.

Mechanism: Under bursty traffic conditions, DSCC continuously monitors fine-grained queue dynamics, particularly the instantaneous queue growth rate, as an indicator of imminent congestion escalation. When a rapid queue buildup is detected, DSCC proactively adapts the PFC threshold by temporarily increasing it, effectively enlarging the buffer's absorption space. This additional buffering headroom allows ECN-marked packets to be generated and propagated back to the sender before the queue reaches the PFC trigger point. Consequently, senders can react by reducing their transmission rates in a timely manner, alleviating congestion at the transport layer and reducing the likelihood of link-layer pause frames. By dynamically reshaping the temporal and spatial relationship between ECN and PFC signals, DSCC avoids over-reliance on reactive pause-based control and achieves a smoother congestion mitigation process, particularly in the presence of microbursts and highly synchronized traffic arrivals common in modern RDMA-based data center workloads.

6.3.1. Initialization

Initialize the network weights θ for the primary Q-network, the target network weights θ^- and the experience replay buffer D .

6.3.2. State observation and action selection

At each time step t , the agent observes the current state s_t , which encapsulates the network parameters such as ECN_c , PFC_c , $q_{len}(t)$, $f_{ECN}(t)$, $\Delta q(t)$, and $I(t)$. The action a_t is selected using an ϵ -greedy policy

$$a_t = \begin{cases} \text{random action,} & \text{with probability } \epsilon, \\ \arg \max_a Q(s_t, a; \theta), & \text{with probability } 1 - \epsilon. \end{cases} \quad (9)$$

The action $a_t = (a_{ECN}, a_{PFC}, \delta_t)$ is defined as in Eq. (3).

where the modulation factor δ_t is computed by Eq. (5)

6.3.3. Action execution and environment feedback

After executing the selected action a_t , the network environment provides

- The immediate reward R_t calculated using Eq. (7).
- The next state s_{t+1} .

The transition is represented as (s_t, a_t, R_t, s_{t+1}) .

6.3.4. Experience replay

The transition (s_t, a_t, R_t, s_{t+1}) is stored in the replay buffer D . This buffer is used to break the temporal correlations between sequential samples by randomly sampling mini-batches for training.

6.3.5. Q-network parameter update

Periodically, a mini-batch $\{(s_j, a_j, R_j, s'_j)\}_{j=1}^N$ is sampled from $\mathcal{D}$. For each sample, the target value is computed as:

$$y_j = R_j + \gamma_d \max_{a'} Q(s'_j, a'; \theta^-). \quad (10)$$

The loss function $L(\theta)$ for the mini-batch is then defined by the mean squared error:

$$L(\theta) = \frac{1}{N} \sum_{j=1}^N (y_j - Q(s_j, a_j; \theta))^2. \quad (11)$$

The gradient of the loss with respect to θ is computed as:

$$\nabla_{\theta} L(\theta) = \frac{2}{N} \sum_{j=1}^N (Q(s_j, a_j; \theta) - y_j) \nabla_{\theta} Q(s_j, a_j; \theta). \quad (12)$$

An optimizer (e.g., stochastic gradient descent or Adam) is then used to update the network parameters θ .

6.3.6. Target network update

To stabilize training, the target network parameters θ^- are updated periodically

$$\theta^- \leftarrow \theta. \quad (13)$$

6.3.7. Iterative learning process

The entire process (from state observation to parameter update) is repeated for each time step, allowing the agent to gradually learn an optimal policy $\pi(s) = \arg \max_a Q(s, a; \theta)$ that maximizes the expected discounted cumulative reward

$$Q^{\pi}(s_t, a_t) = \mathbb{E} \left[\sum_{k=0}^{\infty} \gamma_d^k R_{t+k} \mid s_t, a_t \right]. \quad (14)$$

6.4. Reward function with constraint penalty

To ensure that the agent respects system constraints, we incorporate a penalty term into the reward function. The penalty term P_i is composed of the following components:

6.4.1. ECN-PFC gap constraint

Let $G_{EP} = \sum_{i=1}^N T_{PFC}(i) - T_{ECN}$ represent the ECN-PFC buffer gap. If $G_{EP} < \Delta_{ECN}$, the constraint is violated, and the penalty is computed as:

$$P_{EP}(t) = \max(0, \Delta_{ECN} - G_{EP}) \quad (15)$$

6.4.2. Upstream/downstream threshold constraint

This constraint requires $T_{ECN} < \sum_{i=1}^N T_{PFC}(i)$. If violated, the penalty is:

$$P_{UD}(t) = \max\left(0, T_{ECN} - \sum_{i=1}^N T_{PFC}(i)\right) \quad (16)$$

6.4.3. Downstream shared buffer constraint

Let $T_{OUT-Sharedbf}$ be the total downstream shared buffer, and $T_{Headroom}(i)$ the headroom buffer for each priority queue. The constraint is:

$$\sum_{i=1}^N T_{PFC}(i) < T_{OUT-Sharedbf} - \sum_{i=1}^N T_{Headroom}(i) \quad (17)$$

Let

$$T_{\Delta} = \sum_{i=1}^N T_{PFC}(i) + \sum_{i=1}^N T_{Headroom}(i) - T_{OUT-Sharedbf} \quad (18)$$

If violated, the penalty becomes:

$$P_{SB}(t) = \max(0, T_{\Delta}) \quad (19)$$

6.4.4. Final penalty term

The total penalty is defined as a weighted sum of the individual penalties:

$$P_t = 0.5P_{EP}(t) + 0.3P_{UD}(t) + 0.2P_{SB}(t) \quad (20)$$

6.5. Randomized topology and traffic generation

To guarantee the DSCC agent's ability to generalize beyond a single network configuration, we designed the ns-3 training environment to support a wide spectrum of topologies and traffic conditions. At the start of each learning episode, we sample the network layout-number of switches and hosts, connectivity matrix, and structural paradigm (star, spine-leaf, or fat-tree)-from predefined distributions. Simultaneously, key link attributes (bandwidth between 10Gb/s and 100Gb/s, propagation delay from 10 μ s to 2ms, buffer capacity, ECN marking threshold, and packet-loss probability) are randomized to reflect realistic variability.

Traffic workloads are likewise diversified: the concurrent flow count, per-flow size (following mice-and-elephant distributions), and inter-arrival intervals are drawn from either production trace datasets or synthetic models such as Alistorage and Datamining. By exposing the agent to this randomized ensemble of scenarios, we mitigate overfitting to any particular configuration and foster adaptation to varying congestion levels, link utilizations, and flow dynamics. Consequently, the resulting DSCC policy exhibits robust performance and resilience across a broad range of data-center operating conditions.

6.6. Theoretical feasibility of PFC regulation

On the issue of whether to support dynamic adjustment of PFC threshold, we referred to the settings of Ruijie Network switches. Ruijie supports dynamic allocation of buffer resources, and shared buffer settings are divided into 11 levels [19]. The parameter of the dynamic waterline constant is equal to the number of buffers available in the queue divided by the number of shared buffers remaining. At the same time, Ruijie mentioned that in the actual deployment, there is no fixed value for the PFC threshold. It is necessary to analyze the business model in detail and build a test environment to optimize the waterline and find the most suitable waterline to match the business. Although this article does not directly adopt the static setting method, the design of Ruijie switches shows that it supports dynamic adjustment of PFC thresholds.

7. Evaluation

In this section, we verify the performance of the algorithm through large-scale ns-3 simulation. First, we compare it with existing alternative algorithms in typical traffic datasets, and the results show that it outperforms existing algorithms on typical datasets. Second, we add different levels of incast scenarios to evaluate the comparison of the metrics with existing algorithms.

7.1. Experiment setup

Topology: In this paper, the network topology follows the commonly used Fat-Tree architecture in datacenter networks. It comprises 16 core switches, 20 aggregation switches (Agg) and 20 top-of-rack (ToR) switches, which support a total of 320 servers, with each rack housing 16 servers. The connection between each server and its respective ToR switch operates at a capacity of 100Gbps. The bandwidth between other switches is 400Gbps. All network links have a propagation delay of 1 μ s. The size of the switch buffer is 12MB, which emulates the capabilities of the Broadcom Trident II switch [2].

Workloads: We generate traffic based on widely accepted and recognized *FB_Hadoop* and *Websearch* modes. The two traffic modes reflect different requirements. *WebSearch* seeks high throughput, while

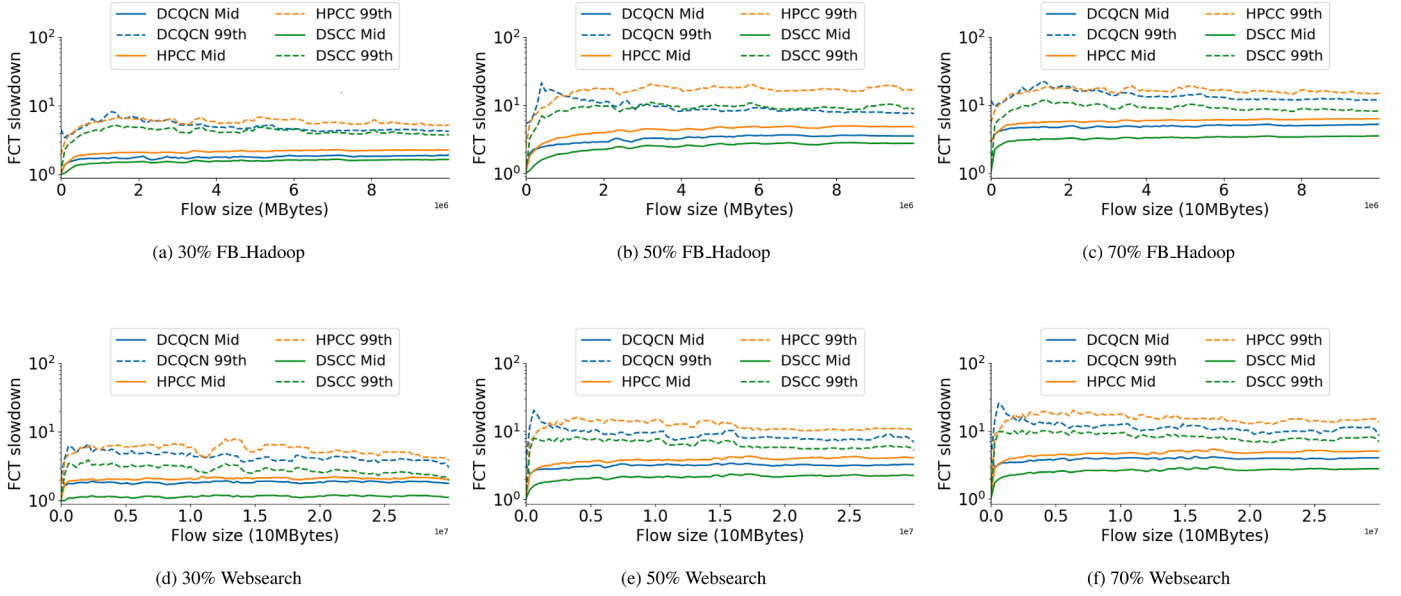


Fig. 6. FCT slowdown of DCQCN, HPCC and DSCC on different workloads.

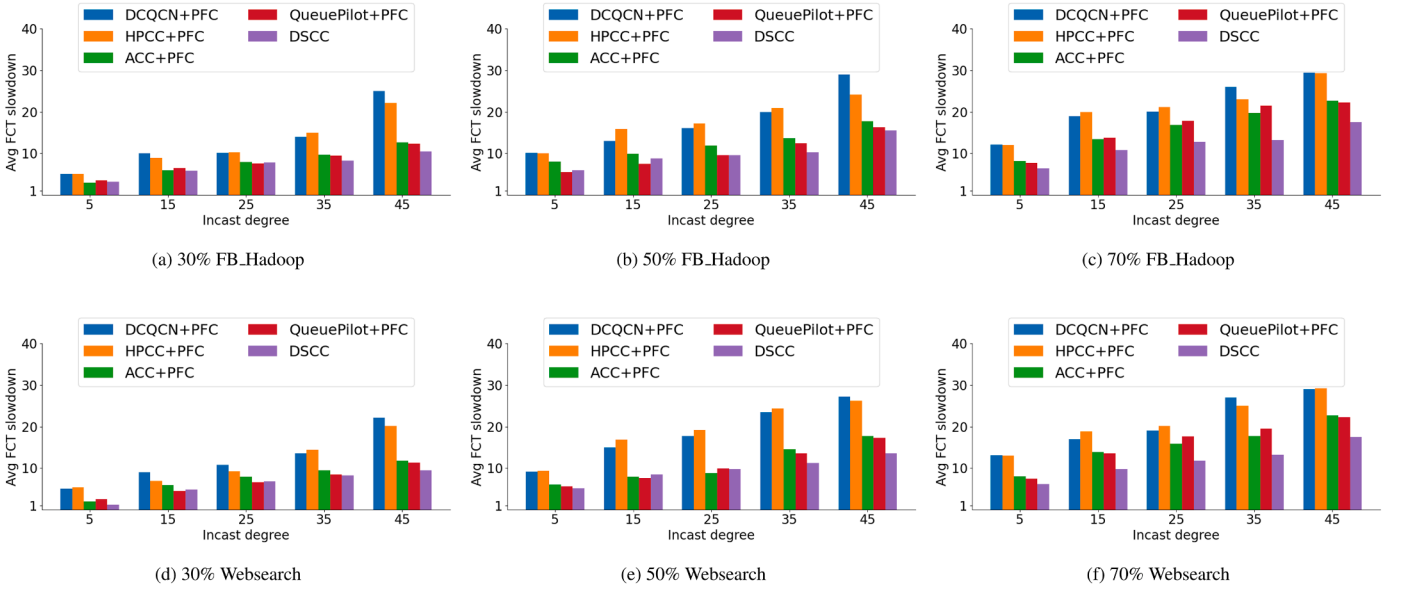


Fig. 7. Avg FCT slowdown of different Algorithms on different workloads.

FB_Hadoop is more sensitive to latency. The network load values are 30, 50, and 70. Subsequently, we introduce incast traffic into both the *Websearch* and *FB_Hadoop* modes to assess the algorithm's robustness under bursty workloads. We simulate incast flows with different degrees of many-to-one relationships to evaluate the performance of DSCC under different degrees of incast flow environments.

Comparisons: We compare the algorithm with existing static ECN algorithms and dynamic ECN algorithms, respectively.

- **Static ECN:** We implement DCQCN [6] and HPCC [2]. PFC, ECN and other related parameter settings refer to the configurations in the DCQCN and HPCC articles.
- **Dynamic ECN:** We implement ACC [9] and QueuePilot [11]. Other parameters related to congestion control are set to the recommended values in the article.

Metrics: We use the following indicators to evaluate our algorithm and other algorithms: FCT slowdown, which is defined as the comple-

tion time of a flow in the current simulation scenario divided by the completion time when it occupies the entire network. The smaller the value, the better the performance. The fraction of PFC pause time, which indicates the ratio of the PFC pause duration to the total transmission duration. The cumulative distribution function (CDF) graph performs a statistical analysis of the queue length.

7.2. Performance

Overall performance: In this section, we evaluate the overall performance of the algorithm. Fig. 6 shows the FCT of the proposed algorithm and other existing algorithms (DCQCN, HPCC) under different workloads (30%, 50%, 70%) for two traffic modes: *Websearch* and *FB_Hadoop*. The results show that DSCC outperforms the existing static ECN algorithm for different types of traffic, and PFC pauses are not triggered in both cases. As shown in the table, the *Websearch* mode generates more large flows, so it is sensitive to throughput. *FB_Hadoop*

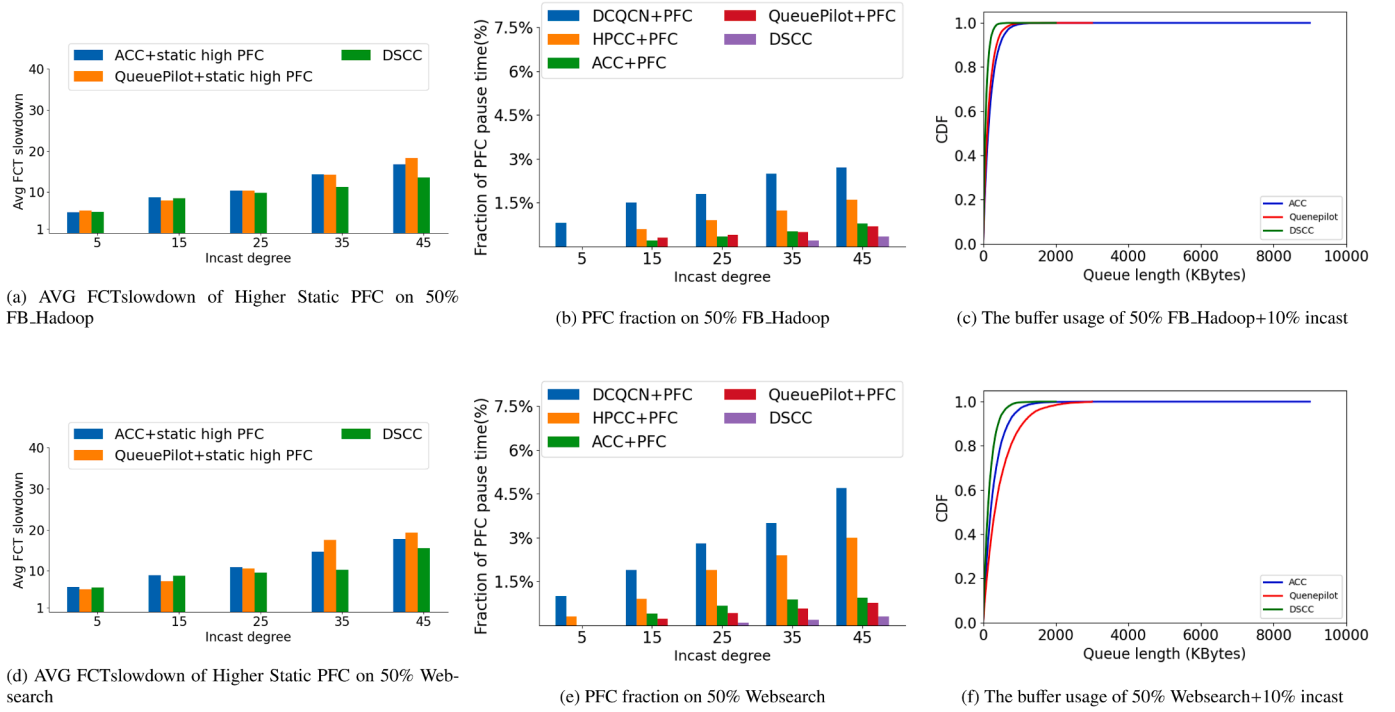


Fig. 8. Fraction of PFC pause time of different Algorithms on different workloads.

generates more small flows, so it is sensitive to latency. In the algorithm in this paper, the ECN threshold is dynamically adjusted by the AI-assisted algorithm for small flows sensitive to latency and large flows sensitive to throughput. Under the influence of the low ECN threshold for small flows and the high ECN threshold for large flows, the optimal ECN is maintained while effectively reducing the flow completion time. As shown in the figure, in different load modes of *Websearch*, compared to DCQCN and HPCC, the average FCT decreased by up to 40.43% and 53.62%, respectively, and in different load modes of *FB_Hadoop*, the average FCT decreased by up to 18.20% and 19.57%. This shows that when PFC is not triggered, DSCC performs better in dynamic adjustment of the ECN than the traditional static threshold method. It also shows that DSCC performs better in the *Websearch* workload.

Incast flow performance: In this section, we evaluate the performance of the algorithm in the incast environment. Based on the above experiments, we added an incast flow with a load of 10% under different workloads of the two traffic modes of *Websearch* and *FB_Hadoop* (30%, 50%, 70%). The size of each burst flow is 1MB. Two types of congestion control algorithms are selected for comparison. We include DCQCN and HPCC for static ECN approaches and adopt ACC and QueuePilot for AI-ECN. The flow control mechanism for comparison is PFC. We randomly select N senders and one receiver to simulate the incast scenarios, where N is 5, 15, 25, 35, 45. Fig. 7a–f are the FCT data for different loads + incast flows in the *Websearch* and *FB_Hadoop* environments, respectively. The experimental results show that in traffic scenarios with different loads of *Websearch* and *FB_Hadoop*, the dynamic collaborative algorithm can effectively reduce the average FCT by up to 40.3% compared to the static ECN algorithm and by up to 11.2% compared to the dynamic ECN algorithm. As N increases, the reduction is greater. Because the algorithm can take advantage of a larger adjustment range when dealing with the intensified queue buildup and bursty traffic typically seen in severe incast scenarios. This expanded flexibility allows the control mechanism to respond more effectively to congestion, resulting in greater improvements in latency and bandwidth utilization.

To further evaluate the effectiveness of DSCC, we compared it with alternatives such as AI ECN and static high-threshold PFC configuration. As shown in Fig. 8a and d, while AI ECN with static high PFC thresh-

old can compete with DSCC at low and medium incast intensities, they fail to maintain low flow completion time (FCT) latency as incast intensity increases. This is mainly due to excessive queue accumulation before triggering PFC, which introduces large queuing delays. In contrast, DSCC dynamically adjusts congestion control behavior based on real-time network status, effectively preventing deep queues while minimizing unnecessary PFC activations.

Fig. 8b and e shows the fraction of PFC pause time. Compared with the traditional methods of changing the PFC pause threshold using the remaining buffer size, DSCC increases the PFC threshold on the original basis and reduces the waste of switch buffer resources. By making different adjustments for different incast environments, DSCC reduces the number of PFC triggers and thus reduces FCT. Fig. 8b shows that under 50% *FB_Hadoop* load, DSCC reduces the PFC pause ratio by 3.1% compared to the static ECN algorithm, and reduces the PFC pause ratio by 1.3% compared to the dynamic ECN algorithm. Fig. 8e shows that under 50% *Websearch* load, DSCC reduces the PFC pause ratio by up to 4.3% compared to the static ECN algorithm and reduces the PFC pause ratio by up to 0.8% compared to the dynamic ECN algorithm. The results indicate that DSCC performs better in burst flow scenarios, particularly in *Websearch* workload compared to *FB_Hadoop*. In addition, when faced with burst flows of different degrees, the DSCC performance is more advantageous when N is larger. In summary, the results show that in the incast environment, that is, when PFC is triggered, the performance of DSCC has significant advantages over other compared static ECN algorithms and AI ECN algorithms.

The CDF plots of buffer usage in Fig. 8c and f show that DSCC achieves superior buffer efficiency compared to ACC and QueuePilot. Unlike AI ECN schemes, DSCC promptly responds to queue buildup and prevents excessive buffering. This leads to tighter queue distributions, reduced queuing delays, and improved latency performance. The consistency of these results across different workloads confirms DSCC's adaptability and real-time responsiveness in dynamic congestion scenarios, making it more resilient to traffic burstiness and incast-induced pressure. Moreover, by proactively maintaining buffer headroom, DSCC significantly reduces the likelihood of triggering PFC pauses, thereby minimizing head-of-line blocking and improving overall flow stability.

This proactive behavior enhances the system's ability to handle sudden surges without compromising performance or reliability.

8. Conclusion

By observing the traffic characteristics of existing data center networks, we conclude the shortcomings of existing congestion control mechanisms in dealing with large-scale incast flow problems. These limitations often result in buffer overflows, excessive queuing delays, and frequent PFC triggering, which ultimately degrade application performance and network stability. In this paper, we propose DSCC, which leverages deep reinforcement learning to make fine-grained decisions that proactively prevent congestion while maintaining high link utilization and low latency. Experimental results across multiple representative data center workloads demonstrate that DSCC significantly reduces flow completion time and lowers the PFC trigger ratio compared to existing schemes. These improvements are consistent and robust across varying incast scales and traffic conditions, confirming that DSCC is both a feasible and highly effective solution for congestion control in RDMA-based data center networks.

CRedit authorship contribution statement

Jianxing Zhuge: Writing – review & editing; **Zeming Gao:** Supervision; **Ye Tian:** Writing – review & editing; **Jun Wang:** Writing – review & editing; **Shaoxuan Yun:** Writing – review & editing; **Xiangyang Gong:** Writing – review & editing.

Declaration of interests

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

The data that has been used is confidential.

Acknowledgement

This work was supported by the National key R&D Program of China under Grant 2023YFB2904100.

References

- [1] C. Hu, B. Li, Distributed inference with deep learning models across heterogeneous edge devices, in: IEEE INFOCOM 2022 - IEEE Conference on Computer Communications, 2022, pp. 330–339.
- [2] Y. Li, R. Miao, H.H. Liu, Y. Zhuang, F. Feng, L. Tang, Z. Cao, M. Zhang, F. Kelly, M. Alizadeh, HPCC: high precision congestion control, in: Proceedings of the ACM Special Interest Group on Data Communication, 2019, pp. 44–58.
- [3] Y. Zhang, Y. Liu, Q. Meng, F. Ren, Congestion detection in lossless networks, in: Proceedings of the 2021 ACM Special Interest Group on Data Communication (SIGCOMM), 2021, pp. 370–383.
- [4] C. Guo, H. Wu, Z. Deng, G. Soni, J. Ye, J. Padhye, M. Lipshteyn, RDMA over commodity ethernet at scale, in: Proceedings of the 2016 ACM SIGCOMM Conference, 2016, pp. 202–215.
- [5] P. Taheri, D. Menikkumbura, E. Vanini, S. Fahmy, P. Eugster, T. Edsall, RoCC: robust congestion control for RDMA, in: Proceedings of the 16th International Conference on Emerging Networking Experiments and Technologies (CoNEXT), 2020, pp. 17–30.
- [6] Y. Zhu, H. Eran, D. Firestone, Congestion control for large-scale RDMA deployments, *ACM SIGCOMM Comput. Commun. Rev.* 45 (4) (2015) 523–536.
- [7] X. Zhong, J. Zhang, Y. Zhang, Z. Guan, Z. Wan, PACC: proactive and accurate congestion feedback for RDMA congestion control, in: Proceedings of the 2022 IEEE International Conference on Computer Communications (INFOCOM), IEEE, 2022.
- [8] H. Chen, FCC: fast source congestion control for ultra-low latency in data center networks, in: IEEE/ACM 32nd International Symposium on Quality of Service (IWQoS), p. 2024.
- [9] S. Yan, X. Wang, X. Zheng, Y. Xia, D. Liu, W. Deng, ACC: automatic ECN tuning for high-speed datacenter networks, in: Proceedings of the 2021 ACM SIGCOMM 2021 Conference, 2021, pp. 384–397.
- [10] Y. Zhang, BBQ: dynamic-buffer-driven automatic ECN tuning in datacenter, in: 2024 IEEE/ACM 32nd International Symposium on Quality of Service (IWQoS), 2024, pp. 1–6.
- [11] M. Dery, O. Krupnik, I. Keslassy, QueuePilot: reviving small buffers with a learned AQM policy, in: IEEE INFOCOM 2023-IEEE Conference on Computer Communications, IEEE, 2023, pp. 1–10.
- [12] X. Li, R-PFC: enhancing RDMA network with restricted and fine-grained PFC, in: IEEE/ACM 32nd International Symposium on Quality of Service (IWQoS), p. 2024.
- [13] M. Long, LSCC: link-segmented congestion control for RDMA in cross-datacenter networks, in: IEEE/ACM 32nd International Symposium on Quality of Service (IWQoS), p. 2024.
- [14] Q. Meng, BCC: re-architecting congestion control in DCNs, in: IEEE INFOCOM 2024-IEEE Conference on Computer Communications, IEEE, 2024.
- [15] W. Bai, K. Chen, S. Hu, K. Tan, Y. Xiong, Congestion control for high-speed extremely shallow-buffered datacenter networks, in: Proceedings of the First Asia-Pacific Workshop on Networking, 2017, pp. 29–35.
- [16] R.B. Basat, S. Ramanathan, Y. Li, G. Antichi, M. Yu, M. Mitzenmacher, Pint: probabilistic in-band network telemetry, in: Proceedings of the ACM SIGCOMM, 2020, pp. 662–680.
- [17] S. Arslan, Y. Li, G. Kumar, N. Dukkupati, Bolt: sub-RTT congestion control for ultra-low latency, in: USENIX NSDI, 2023, pp. 219–236.
- [18] Z. Chen, M. Zhang, G. Li, M. Xu, Poster: chameleon: automatic and adaptive tuning for DCQN parameters in RDMA networks, in: Proceedings of the ACM SIGCOMM 2023 Conference, 2023, pp. 1091–1093.
- [19] V. Addanki, M. Apostolaki, M. Ghobadi, S. Schmid, L. Vanbever, ABM: active buffer management in datacenters, in: Proceedings of the ACM SIGCOMM 2022 Conference, 2022, pp. 36–52.
- [20] Y. Su, QRED: a Q-learning-based active queue management scheme, *JIT* 19 (4) (2018) 1169–1178.
- [21] J. Hu, Load balancing with multi-level signals for lossless datacenter networks, *IEEE/ACM Trans. Netw.* 32 (2024) 2736–2748.
- [22] J. Hu, Enhancing load balancing with in-network recirculation to prevent packet reordering in lossless data centers, *IEEE/ACM Trans. Netw.* 32 (2024) 4114–4127.
- [23] J. Hu, Traffic-aware load balancing based on deep reinforcement learning in cloud-based industrial data centers, *IEEE Trans. Ind. Inf.* 21 (2025) 9229–9239.
- [24] J. Hu, ARS: adaptive routing system for heterogeneous traffic in industrial data centers, *IEEE Trans. Ind. Inf.* 21 (2025) 6495–6504.
- [25] J. Hu, SRCC: sub-RTT congestion control for lossless datacenter networks, *IEEE Trans. Ind. Inf.* 21 (2024) 2799–2808.
- [26] J. Tang, ABS: Adaptive Buffer Sizing Via Augmented Programmability with Machine Learning, *IEEE Infocom*, 2022.
- [27] G. Zeng, L. Chen, B. Yi, K. Chen, Cutting tail latency in commodity datacenters with cloudburst, in: IEEE INFOCOM 2022 - IEEE Conference on Computer Communications, 2022.
- [28] A. Roy, H. Zeng, J. Bagga, G. Porter, A.C. Snoeren, Inside the social network's (datacenter) network, in: Proceedings of the 2015 ACM Conference on Special Interest Group on Data Communication, 2015, pp. 123–137.
- [29] NS-3 simulator, 2024. <https://www.nsnam.org/>.
- [30] H. Yin, P. Liu, K. Liu, L. Cao, L. Zhang, Y. Gao, X. Hei, ns3-ai: fostering artificial intelligence algorithms for networking research, in: Proceedings of the 2020 Workshop on ns-3, 2020, pp. 57–64.
- [31] Q. Meng, F. Ren, Lightning: a practical building block for RDMA transport control, in: 2021 IEEE/ACM 29th International Symposium on Quality of Service (IWQoS), 2021, pp. 1–10.
- [32] A brief analysis of MMU waterline settings in RDMA networks, 2018. <https://www.ruijie.com.cn/jszl/61714/>.